Amstrad CPC · Z80

Manual de uso de GlasnostFWK

Construcción modular de juegos y pruebas técnicas con CPCTelera Next

Edición: 25 de octubre de 2026 · Idioma: castellano

1. Qué es y qué cubre este manual

GlasnostFWK es un framework modular en ensamblador Z80 para Amstrad CPC. Un proyecto selecciona únicamente los módulos que necesita mediante un manifiesto JSON; el constructor prepara un árbol aislado y CPCTelera Next genera los medios CPC.

Este manual describe el estado comprobado del repositorio: núcleo de sistema, entrada, pool, vídeo Mode 0, texto, ventanas restaurables, movimiento compacto, perfiles separados de plataforma y física 8.8, animación, audio AKG, bancos de 128 KB, carga AMSDOS y ZX7B. No presupone una estructura universal de entidad ni contiene reglas o recursos de los juegos de origen.

> **Límites importantes.** No hay verificación en hardware real ni garantía temporal global de 50 Hz. El perfil 160×288 tiene comprobado su modelo de memoria, pero su señal CRTC continúa siendo experimental. La física completa 8.8 todavía no forma parte del framework.

Índice

1. Qué es y qué cubre
2. Requisitos
3. Primeros pasos
4. Estructura del repositorio
5. Crear un proyecto o ejemplo
6. Referencia de módulos
7. Perfiles y recursos
8. Ejemplos incluidos
9. Pruebas y emulación
10. Distribución
11. Diagnóstico
12. Mantener este manual

2. Requisitos

- Python 3 y GNU Make.
- CPCTelera Next en la revisión fijada `11e61fe30175ad7b328de9bcd859c46fd61b26a5`, con SDCC Z80 4.5.0 y sus herramientas construidas.
- Ruta predeterminada: `/config/workspace/cpctelera_next/cpctelera`. Puede sustituirse con `CPCT_NEXT_PATH` o con `--cpct-path`.
- AmSpiriT-Lite 1.15.3 solo para ejecutar la batería de emulador existente.
- WeasyPrint 70.0 solo para regenerar este manual en PDF.

```
export CPCT_NEXT_PATH=/ruta/a/cpctelera
python3 tools/build_examples.py --cpct-path "$CPCT_NEXT_PATH" minimal
```

La compatibilidad con instalaciones distintas de la revisión fijada no está confirmada. Si se cambia, hay que volver a compilar y ejecutar las pruebas.

3. Primeros pasos

3.1 Construir todo

```
cd /config/workspace/GlasnostFWK
make generate
make examples
make test
```

`make generate` produce las tablas de vídeo y recursos. `make examples` construye los doce consumidores y crea un único disco de pruebas en `.build/tests/GlasnostFWK-tests.dsk`. `make test` ejecuta la suite host y vuelve a construirlos, pero no abre el emulador.

3.2 Construir un único ejemplo

```
python3 tools/build_examples.py minimal
python3 tools/build_examples.py animation vertical
```

Los resultados quedan en `.build/<nombre>/`. Según el manifiesto aparecen:

- `obj/<nombre>.bin`: binario AMSDOS.
- `<nombre>.cdt`: cinta, salvo ejemplos que declaren solo DSK.
- `selected-modules.json`: prueba de qué módulos entraron realmente.

`tools/build_examples.py` es la herramienta de bajo nivel y CPCTelera puede dejar un DSK intermedio por ejemplo. La política pública de `make examples` los sustituye por el único DSK común y deja su catálogo reproducible en `.build/tests/catalog.json`.

3.3 Probar manualmente en un CPC o emulador

1. Ejecute `make examples`.
2. Inserte `.build/tests/GlasnostFWK-tests.dsk` o el CDT específico en el CPC/emulador.
3. En DSK ejecute `CAT` y cargue el binario cuyo nombre coincide con el ejemplo; por ejemplo, `RUN"VERTICAL"`. Los nombres de más de ocho caracteres se ajustan a AMSDOS: `resources128` es `RESOURCE.BIN` y `animation` es `ANIMATIO.BIN`.
4. Para la demostración jugable `vertical`, mantenga CursorRight y recoja los tres puntos.

> Los smoke automáticos no se limitan a comprobar que el emulador sigue abierto: leen RAM, memoria de vídeo, teclado y, en audio, registros PSG.

4. Estructura del repositorio

Ruta	Responsabilidad
`src/core`	Sistema, entrada y pool genérico.
`src/video`	Mode 0, CRTC, direcciones y limpieza visible.
`src/ui`	Texto Mode 0 y ventanas con fondo restaurable.
`src/audio`	Adaptador opcional Arkos Tracker 3 AKG.
`src/resources`	Bancos 128 KB, AMSDOS y ZX7B.
`src/extensions`	Movimiento, física especializada y animación opcionales.
`examples`	Consumidores independientes y sus fuentes de juego.
`profiles`, `manifests`	Geometría y recursos declarativos.
`generated`	Tablas producidas por herramientas; no son la fuente canónica.
`tools`, `tests`	Construcción, generación, paquetes y validación.

La dirección de dependencias es `juego → extensión opcional → núcleo → CPCTelera/hardware`. Ningún archivo de `src/` puede importar un ejemplo o símbolos de juego.

5. Crear un proyecto o ejemplo

5.1 Directorio mínimo

```
examples/mi_juego/
├── example.json
└── main.s
```

El manifiesto mínimo selecciona un perfil, módulos y fuentes propias:

```
{
  "name": "mi_juego",
  "profile": "standard-160x200.json",
  "modules": [
    "core/system.s",
    "core/input.s",
    "video/video.s"
  ],
  "game_sources": ["main.s"]
}
```

Después añada el nombre a `EXAMPLES` en el `Makefile` si debe formar parte de la construcción completa y del paquete oficial.

5.2 Campos del manifiesto

Campo	Uso
`name`	Nombre del proyecto y de sus artefactos.
`profile`	JSON de vídeo situado en `profiles/`.
`modules`	Fuentes del framework relativos a `src/`.
`game_sources`	Fuentes `.s` / `.inc` del consumidor. Debe incluir `main.s`.
`cpct_modules`	Fuentes concretos de CPCTelera que deben compilarse, por ejemplo el player AKG.
`media`	Lista con `"dsk"`, `"cdt"` o ambos. Por defecto genera ambos.
`disk_files`	Payloads adicionales para DSK: fuente, codificación, nombre AMSDOS, símbolo y compresión.

Las rutas absolutas, recorridos `..`, ficheros inexistentes, nombres AMSDOS inválidos y símbolos duplicados se rechazan antes de construir.

5.3 Fuente principal

```
.module mi_juego
.globl fwk_system_init, fwk_system_wait_frame
.globl fwk_video_init

.area _CODE
_main::
    call fwk_system_init
    call fwk_video_init
loop:
    call fwk_system_wait_frame
    jr loop
```

Declare con `.globl` únicamente las funciones que use. Los módulos ausentes del manifiesto no se copian ni enlazan.

6. Referencia de módulos

Los contratos indican entradas, salidas y registros alterados. El llamador debe respetarlos; el framework no inserta comprobaciones de límites en tiempo de ejecución salvo donde se indique.

6.1 Sistema — `core/system.s`

- `fwk_system_init`: desactiva firmware, instala IRQ y deja el callback nulo. Altera `AF,BC,DE,HL`.
- `fwk_system_wait_frame`: espera VSYNC. Altera `AF,BC,DE,HL`.
- `fwk_system_set_irq_callback`: recibe `HL=callback|0`. El callback se ejecuta una vez por cada seis IRQ, después del escaneo del teclado.
- Estado público: `fwk_system_irq_count`, `fwk_system_frame_count` y el puntero de callback.

No cambie el callback mientras está activo sin deshabilitar interrupciones. Un callback debe preservar lo que necesite el programa principal.

6.2 Entrada — `core/input.s`

Admite hasta ocho acciones lógicas y separa pulsación sostenida, flanco de entrada y flanco de salida.

```
bindings:
    .dw Key_CursorRight
    .db 0x01
    .dw Key_CursorLeft
    .db 0x02
    .dw 0
    .db 0
```

- `fwk_input_init`: borra el estado.
- `fwk_input_set_bindings`: `HL=tabla`. La tabla debe persistir y termina con máscara cero.
- `fwk_input_update`: actualiza `fwk_input_held`, `fwk_input_pressed` y `fwk_input_released`. Altera `AF,BC,DE,HL,IY`.

6.3 Pool compacto — `core/pool.s`

Incluya `core/pool.inc` y cree un descriptor de cinco bytes:

```
entities: .ds 8 * 5
FWK_POOL_DESCRIPTOR entity_pool, entities, 8, 5
```

- `fwk_pool_init`: `IX=descriptor`; count pasa a cero.
- `fwk_pool_get`: `A=índice`; devuelve `HL=registro` o carry si no existe.
- `fwk_pool_alloc`: devuelve un slot sin inicializar o carry si está lleno.
- `fwk_pool_remove`: elimina `A=índice` y compacta con `LDIR`.
- `fwk_pool_foreach`: `DE=callback`; el callback recibe `IY=registro`.

`foreach` usa un `CALL` automodificable: no es reentrante ni debe reentrarse desde IRQ. El pool no interpreta los bytes de cada registro.

6.4 Vídeo Mode 0 — `video/video.s`

- `fwk_video_init`: selecciona Mode 0.
- `fwk_video_configure_crtc`: aplica la tabla CRTC del perfil. Usarlo no convierte un perfil experimental en verificado.
- `fwk_video_get_ptr`: `HL=y` de 16 bits y `C=x` en bytes; devuelve `HL=puntero`.
- `fwk_video_plot_byte`: añade `A=byte empaquetado Mode 0`.
- `fwk_video_clear_visible`: `A=patrón`; limpia solo regiones visibles y conserva huecos.

`fwk_video_width_bytes` y `fwk_video_height` exponen la geometría activa para módulos de validación. El ancho estándar es 80 bytes, equivalentes a 160 píxeles en Mode 0. El llamador debe mantener las coordenadas dentro del perfil.

6.5 Texto — `ui/text.s`

- `fwk_text_set_colors`: `L=foreground`, `H=background`, ambos 0..15.
- `fwk_text_draw`: `IY=cadena ASCII NUL`, `HL=puntero de vídeo`.

El adaptador usa las rutinas Mode 0 de CPCTelera. Su implementación subyacente es automodificable y no puede ejecutarse desde ROM.

6.6 Ventanas restaurables — `ui/window.s`

```
message_window:
    .db 10       ; x en bytes Mode 0
    .dw 40       ; y de 16 bits
    .db 20, 16   ; ancho y alto
    .db 0xFF     ; patrón de borde
    .db 0x00     ; patrón de relleno

window_buffer: .ds 20 * 16
```

- `fwk_window_capture`: `IX=descriptor`, `DE=buffer`; guarda `width*height` bytes.
- `fwk_window_draw`: `IX=descriptor`; dibuja borde de un byte y relleno.
- `fwk_window_restore`: `IX=descriptor`, `HL=buffer`; repone exactamente la captura.

Carry indica geometría inválida. El módulo usa las dimensiones y tabla de líneas del perfil; no contiene texto, entrada, espera ni un buffer global. El llamador debe reservar suficiente memoria y conservar la geometría entre captura y restauración. El contrato detallado está en `docs/windows.md`.

6.7 Movimiento compacto — `extensions/motion_u8.s`

`fwk_motion_u8_step` recibe en `IY` un registro de cinco bytes `x,y,vx,vy,flags`. Aplica velocidades con signo de 8 bits y satura en `x=0..79`, `y=0..199`. Altera `AF`.

Este perfil no es una entidad universal y no representa física 8.8. Es reentrante si el registro no se comparte.

6.8 Física de plataforma — `extensions/physics_platform.s`

`fwk_physics_platform_step` recibe `IY=estado` e `IX=configuración`. El estado de siete bytes contiene `x,y,vx,vy,width,height,flags`; la configuración define ancho del mundo, coordenada inclusiva del suelo, gravedad y caída máxima.

Aplica velocidad byte con signo, límites horizontales, gravedad, techo, suelo y el bit `FWK_PLATFORM_AIRBORNE`. No conoce mapas, tiles, pools ni sprites. Altera `AF,BC`, preserva `IX/IY` y es reentrante si el estado no se comparte. El consumidor debe usar dimensiones no nulas, `width <= world_width` y parámetros que no desborden la velocidad con signo.

6.9 Física 8.8 — `extensions/physics_8_8.s`

`fwk_physics_8_8_step` recibe `IY=estado` e `IX=descriptor`. Posición y velocidad son palabras 8.8 little-endian; el noveno byte conserva el resto sub-LSB de fricción. El descriptor aporta límites inclusivos y fricción de `0..255`.

La fricción sigue el vector para no curvar diagonales al detener antes el eje menor. Después integra y rebota por eje. Altera `AF,BC,DE,HL`, preserva `IX/IY` y no es reentrante porque usa cuatro palabras internas. No resuelve colisiones entre cuerpos ni evita por sí sola atravesar geometría con velocidades excesivas. Véase `docs/physics-profiles.md`.

6.10 Animación — `extensions/animation.s`

```
walk_animation:
    .db 3               ; número de frames
    .db 4               ; ticks por frame
    .dw frame0, frame1, frame2

animation_state: .ds FWK_ANIM_STATE_SIZE
```

- `fwk_anim_clear`: `IX=estado`; desactiva y devuelve `HL=0`.
- `fwk_anim_set`: `IX=estado`, `HL=descriptor|0`. Carry indica que cambió. Repetir el descriptor conserva índice y temporizador.
- `fwk_anim_tick`: consume un tick decidido por el juego y devuelve el puntero de frame. Carry indica avance.

El descriptor y los frames deben seguir visibles. Count y duración deben ser mayores que cero. El juego decide cuándo avanzar, qué descriptor elegir y cómo dibujarlo.

6.11 Audio AKG — `audio/akg.s`

El manifiesto también debe incluir `audio/arkosTracker_PlayerAkg.s` en `cpct_modules`, y el juego debe aportar su canción y tabla de efectos exportadas por Arkos Tracker.

1. `fwk_audio_init_music`: `HL=canción`, `A=subsong`.
2. `fwk_audio_init_sfx`: `HL=tabla de efectos`.
3. `fwk_audio_start`: instala el tick AKG en el callback del sistema.
4. `fwk_audio_play_sfx`: `A=id`, `B=volumen invertido`, `C=canal 0..2`.
5. `fwk_audio_stop`: para el player y retira el callback.

El player y sus datos deben permanecer visibles durante la IRQ. El coste total variable del player no está medido; no se promete 50 Hz para cualquier canción o escena.

6.12 Bancos de 128 KB — `resources/bank.s`

- `fwk_bank_init`: detecta RAM adicional. Devuelve `A=1/NZ` o `A=0/Z` y actualiza `fwk_bank_has_128k`.
- `fwk_bank_copy_from`: `A=banco 0..3`, `HL=origen bancado`, `DE=destino normal`, `BC=tamaño`.
- `fwk_bank_copy_to`: mismo contrato, con origen normal y destino bancado.

El lado bancado debe estar entre `0x4000` y `0x7FFF`. El stub ocupa `0x0200`, prueba `0x7FF0` y deja las interrupciones habilitadas. Carry indica banco inválido.

6.13 Carga AMSDOS — `resources/disk.s`

- `fwk_disk_init`: reconecta AMSDOS. Debe llamarse antes de `fwk_system_init`.
- `fwk_disk_load`: `HL=nombre`, `B=longitud`, `DE=destino`. Carry y `BC=longitud lógica` indican apertura correcta.

Usa `0xF800..0xFFFF` como buffer de cabecera y no es reentrante. El llamador debe verificar longitud y contenido; este backend no ofrece por sí solo integridad.

6.14 Descompresión ZX7B — `resources/zx7b.s`

`fwk_zx7b_unpack` recibe `HL=último byte comprimido` y `DE=último byte del destino`. Escribe hacia atrás mediante `cpct_zx7b_decrunch_s_asm` y altera `AF,BC,DE,HL`.

No comprueba límites ni corrupción. Para archivos declarados en `disk_files`, el constructor genera constantes `FWK_DISK_*_SIZE` y `FWK_DISK_*_UNPACKED_SIZE`.

7. Perfiles y recursos

7.1 Perfil estándar 160×200

`profiles/standard-160x200.json` define Mode 0, 80 bytes por línea, 200 líneas, pantalla en `0xC000` y pila reservada en `0xB800..0xBFFF`. Es el perfil usado por los ejemplos.

7.2 Perfil experimental 160×288

`profiles/experimental-160x288.json` usa coordenadas verticales de 16 bits y dos segmentos de vídeo. Se han comprobado 23.040 direcciones únicas y el cruce de memoria; la temporización CRTC no se ha validado visualmente en varios CRTC ni en hardware.

7.3 Generar tablas

```
python3 tools/gen_video.py profiles/standard-160x200.json --out-dir
generated
python3 tools/gen_resources.py manifests/resources128.json --output
generated/resources128.inc
```

Edite los JSON fuente y regenere. No edite las tablas de `generated/` a mano. El generador de recursos rechaza solapamientos, destinos reservados, bancos no disponibles y alineaciones incorrectas.

7.4 Archivos adicionales en DSK

```
"disk_files": [{
  "source": "mapa.bin",
  "encoding": "raw",
  "name": "MAPA.BIN",
  "symbol": "mapa",
  "compression": "zx7b"
}]
```

`name` debe cumplir AMSDOS 8.3 en mayúsculas. `encoding` admite `raw` y `hex`; `compression`, `none` y `zx7b`. Los payloads se incorporan al DSK, no al ejecutable.

8. Ejemplos incluidos

Ejemplo	Qué demuestra	Medios
`minimal`	Sistema, entrada y vídeo mínimos.	BIN, DSK común, CDT
`board`	Entrada, texto y composición sencilla de tablero.	BIN, DSK común, CDT
`platform`	Pool y perfil de movimiento compacto.	BIN, DSK común, CDT
`gravity`	Gravedad, salto, límites y aterrizaje con velocidad byte.	BIN, DSK común, CDT
`physics88`	Integración 8.8, fricción vectorial y rebote.	BIN, DSK común, CDT
`video`	Direcciones y limpieza de regiones visibles.	BIN, DSK común, CDT
`resources128`	Detección y copia bancaria, con fallback de 64 KB.	BIN, DSK común, CDT
`diskload`	Carga AMSDOS cruda y ZX7B con comparación byte a byte.	BIN, DSK común
`audio`	Música AKG, stop/restart y SFX.	BIN, DSK común, CDT
`animation`	Tres frames, periodo exacto, pausa y reanudación.	BIN, DSK común, CDT
`window`	Validación, captura, caja, texto, restauración y reapertura.	BIN, DSK común, CDT
`vertical`	Escena jugable: entrada, movimiento, pool, render y victoria.	BIN, DSK común, CDT

`vertical` es la única escena jugable. Los otros once son pruebas técnicas deliberadamente pequeñas.

9. Pruebas y emulación

9.1 Suite local

```
make test
make report
```

La suite comprueba generadores, límites de vídeo, seguridad de manifiestos, frontera de dependencias, nombres AMSDOS y colisiones del DSK común, paquete, cobertura de este manual y compilación. `make report` actualiza `docs/build-sizes.md` desde los mapas SDCC.

9.2 Smoke completo en CPC 6128

```
python3 tests/emulator_smoke.py
```

9.3 Casos específicos en CPC 464

```
python3 tests/emulator_smoke.py --model 464 resources128
python3 tests/emulator_smoke.py --model 464 diskload
python3 tests/emulator_smoke.py --model 464 audio
python3 tests/emulator_smoke.py --model 464 animation
python3 tests/emulator_smoke.py --model 464 gravity physics88
python3 tests/emulator_smoke.py --model 464 window
python3 tests/emulator_smoke.py --model 464 vertical
```

Un build correcto, una prueba host correcta, una ejecución emulada y una prueba en hardware son evidencias distintas. Consulte `docs/validation.md` para ver cuáles se han realizado realmente.

10. Crear un paquete de distribución

```
make release VERSION=0.2.0
(cd dist && sha256sum --check SHA256SUMS)
```

El empaquetador construye los ejemplos y crea en `dist/` un ZIP con los doce BIN, once CDT y un solo `tests/GlasnostFWK-tests.dsk`, además de manifiesto, instrucciones y `SHA256SUMS`. Crear este paquete local no publica automáticamente una GitHub Release.

Use una versión con forma `X.Y.Z` y sufijo opcional válido. La release pública estable confirmada en la fecha de este manual es `v0.1.0`; el desarrollo posterior puede contener funciones aún no incluidas en esa release.

11. Diagnóstico rápido

Síntoma	Comprobación
No se encuentra CPCTelera.	Revise `CPCT_NEXT_PATH` o use `--cpct-path`; confirme que existen las herramientas y `cpctelera.lib`.
El módulo no se enlaza.	Añádalo a `modules`; para fuentes del juego use `game_sources`; para el player externo use `cpct_modules`.
Entrada siempre vacía.	Llame a `fwk_system_init`, instale una tabla persistente terminada en cero y ejecute `fwk_input_update` cada frame.
El archivo DSK no carga.	Llame a `fwk_disk_init` antes de desactivar firmware; compruebe nombre 8.3, longitud y destino.
ZX7B corrompe memoria.	Verifique últimos bytes de origen/destino y tamaños generados. No hay comprobación interna de límites.
Audio no avanza.	Inicialice música, mantenga player/datos visibles y llame a `fwk_audio_start` después del sistema.
La animación se reinicia.	No borre el estado. `fwk_anim_set` es idempotente solo al recibir el mismo descriptor y bloque de estado.
La ventana no restaura bien.	Reserve exactamente `width*height` bytes o más y no cambie descriptor, buffer ni perfil entre captura y restauración.
Falla `make test` por el manual.	Documente el nuevo ejemplo/módulo en este HTML y ejecute `make manual`.

12. Mantener y ampliar este manual

Este documento forma parte del contrato del proyecto. Siempre que se añada, cambie o retire una funcionalidad:

1. Actualice la explicación y el contrato correspondiente en `docs/manual.html`.
2. Si aparece un módulo seleccionable, marque su sección con `data-module="ruta/del/modulo.s"`.
3. Si aparece un ejemplo, añada una fila con `data-example="nombre"`.
4. Regenerar PDF y checksum: `make manual`.
5. Ejecute `make test` y registre el resultado real en `PROJECT_STATUS.md`.

```
python3 -m venv .venv-docs
.venv-docs/bin/python -m pip install -r docs/requirements-manual.txt
make manual PYTHON=.venv-docs/bin/python
make test
```

La prueba `tests/test_manual.py` compara los ejemplos y módulos seleccionados con las marcas del manual, valida que el PDF existe y comprueba que `manual.sha256` corresponde al HTML actual. Así, un cambio funcional obliga a revisar esta documentación antes de integrar.

Fuentes complementarias: `docs/architecture-and-api.md`, `docs/validation.md`, `docs/provenance.md` y `PROJECT_STATUS.md`. Ante discrepancias, los contratos del código y la evidencia reproducible prevalecen sobre una descripción antigua.